数据流驱动的开源高层次综合：Google XLS 技术调研

技术调研, Claude (AI Assistant)

2026 年 4 月 10 日

摘要

随着摩尔定律趋缓，专用硬件加速器的开发效率成为制约后摩尔时代计算性能提升的关键瓶颈。传统基于 C/C++的高层次综合（High-Level Synthesis, HLS）工具虽然降低了硬件设计门槛，但其隐含的冯 · 诺依曼计算模型与全局可变状态假设，在描述并行硬件时存在根本性失配。Google 于 2020 年开源的 XLS（Accelerated HW Synthesis）项目采用数据流计算模型为核心，提出了一套面向末代摩尔定律时代（End of Moore's Law）的硬件软件开发工具包（SDK）。本文系统梳理 XLS 的核心技术架构，重点分析其专用数据流语言 DSLX、Sea-of-Nodes 中间表示（IR）、基于 Token 的隐式同步机制，以及配套的调度与优化流程，并对比传统 HLS 工具阐述其设计优势与适用场景。

关键词：高层次综合；数据流计算；硬件加速器；Kahn 进程网络；编译器中间表示

1 引言

高层次综合技术的发展可追溯到 20 世纪 90 年代，其核心目标是将算法描述自动转换为寄存器传输级（Register Transfer Level, RTL）实现，从而缩短专用集成电路（ASIC）与现场可编程门阵列（FPGA）的设计周期。主流商业工具如 Xilinx Vivado HLS 和 Intel HLS Compiler 采用 C/C++作为输入语言，通过添加 pragma 指令来指导编译器进行流水线展开、存储分区与接口综合 [1]。然而，C 语言本质上是围绕串行执行与全局可变状态设计的，这种冯 · 诺依曼风格的计算假设与硬件中" 所有运算并行发生" 的物理现实存在深刻的语义鸿沟。编译器不得不通过控制流图（Control-Flow Graph, CFG）提取数据依赖、推测内存别名关系，并在循环边界插入复杂的状态机，导致生成的硬件往往难以达到手工设计 RTL 的性能与资源效率。

数据流计算模型为此提供了一条根本性的替代路径。早在 20 世纪 70 年代，Gilles Kahn 就提出了以异步进程通过网络中的无界 FIFO 通道进行通信的形式化语义，即 Kahn 进程网络（Kahn Process Networks, KPN）[2]。在 KPN 中，计算单元仅在输入数据可用时才被" 激发"（fire），运算之间的同步由数据依赖隐式决定，无需显式时钟或全局控制单元。这种模型天然契合数字电路的并行特性：逻辑门在输入稳定后自然产生输出，信号沿着导线传播并驱动下游门电路。然而，早期的数据流架构实践——如 Manchester 数据流机和 MIT 标记令牌（Tagged-Token）数据流机——由于细粒度同步开销过大、局部性利用不足以及控制结构表达困难，未能在商用领域取得广泛成功 [3]。

2000 年以后，粗粒度可重构架构（CGRA）和数据流风格的深度学习加速器重新激发了学术界和工业界对这一范式的兴趣。Google 于 2020 年 5 月在 GitHub 上开源的 XLS 项目，将数据

流思想系统性地应用于高层次综合领域 [4]。XLS 不追求在现有 C/C++编译器上修修补补，而是从头设计了一门专用的数据流领域特定语言 DSLX（Domain Specific Language: X），并围绕单一" Sea-of-Nodes" 中间表示构建完整的编译、优化、调度和 RTL 生成工具链。根据 XLS 项目负责人 Chris Leary 在加州大学圣克鲁兹分校 2024 年冬季课程 CSE228A（敏捷硬件设计）上的系统介绍 [5]，XLS 的核心设计哲学可概括为：让软硬件工程师在共享的源代码制品上协作，通过自动化与机器周期来加速专用硬件 IP 的开发，并保证同一份设计既能作为主机软件高效执行，也能生成功能等价的可综合硬件。

本文从 DSLX 语言的数据流语义、XLS IR 的架构设计、编译优化流程，以及调度与代码生成四个维度展开技术分析，最后讨论当前局限与未来研究方向。

2 DSLX 语言：面向硬件的数据流抽象

2.1 语言定位与核心特征

DSLX 是一门受 Rust 语法启发的函数式、不可变、表达式导向的数据流语言 [6]。与 SystemC 或 C++-with-pragmas 不同，DSLX 从根本上摒弃了全局可变状态和顺序执行线程的假设。所有对象的大小在编译时即可确定（arbitrary bitwidths, entirely fixed-size objects），这确保了生成的硬件面积和时序完全可预测。

表 1 对传统 HLS 与 XLS DSLX 的关键特征进行了对比。可以看出，XLS 在语言层面的选择明确服务于" 硬件亲和"（hardware-oriented）这一核心目标：没有指针、没有动态内存分配、没有未定义行为，函数调用图完全静态可分析。这些约束虽然限制了 DSLX 作为通用编程语言的能力，但却极大地简化了编译器到 RTL 的可靠映射。

表 1: 传统 C-based HLS 与 XLS DSLX 的特征对比

特征维度	**传统 HLS（Vivado/Vitis）**	**XLS DSLX**
计算模型	冯 · 诺依曼顺序执行	Kahn 进程网络/数据流
控制流表示	CFG 分支与循环	`Select` 多路选择器
状态与并发	全局变量+pragma	`proc`+`channel`
同步机制	显式时钟/握手	Token 驱动的隐式同步
位宽支持	固定标准类型	任意位宽（`uN` / `sN`）
执行保证	软件仿真与 RTL 可能不等价	保证语义等价+内置形式验证

2.2 位宽与位提取语法

与 C/C++等传统语言仅能依靠移位和掩码手动构造位提取不同，DSLX 对 `bits` 类型提供了**原生位切片语法**，开发者无需调用任何内建函数即可直接提取特定位段。DSLX 采用类似 Python 的切片规则，支持两种互补形式：

（1）范围切片 `x[start:end]`。提取从 `start`（含）到 `end`（不含）之间的位，索引从 LSB（右侧）为 `0` 开始计数，且支持负索引（相对于 MSB）。由于范围两端必须是编译期常量，编译器可直接推断结果位宽。例如，`x[8:16]` 提取第 8 15 位，返回 `u8` 类型；`x[-8:]` 提取最高 8 位。

(2) 宽度切片 `x[start +: Type]`。从 `start` 位开始提取固定宽度的位段，`start` 可以是运行期变量。结果类型显式指定（如 `u8`、`s16`），编译器将其 lower 为 IR 的 `bit_slice` 节点。例如，`x[offset +: u8]` 等价于从动态偏移处提取 8 位。

代码清单 1 展示了 DSLX 位提取语法的典型用法，以及与 C 语言手动 `shift+mask` 模式的对比。

```
let x = u32:0xDEAD_BEEF;

// 范围切片（编译期常量边界）
let low_byte  = x[0:8];     // u8:0xEF
let mid_nibble = x[8:12];   // u4:0x0E
let top_byte  = x[-8:];     // u8:0xDE

// 宽度切片（运行期起始位）
let offset = u5:8;
let dynamic_byte = x[offset +: u8];  // u8:0xBE
```

Listing 1: DSLX 原生位提取语法示例

```
uint32_t x = 0xDEAD_BEEF;

// 提取位 8~15：只能依赖手动移位+掩码
uint8_t byte1 = (x >> 8) & 0xFF;

// 动态偏移更加繁复，且依赖编译器后端识别优化
#define EXTRACT_FIELD(val, start, len) \
    (((val) >> (start)) & ((1U << (len)) - 1U))
uint8_t dynamic = EXTRACT_FIELD(x, offset, 8);
```

Listing 2: C 语言中无原生位提取语法的等价实现

DSLX 的原生位切片不仅提高了代码可读性，更重要的是它直接映射到 XLS IR 的 `bit_slice` 操作，避免了对 barrel shifter 的依赖。然而，正如第 4.3 节所分析的，当前 `arith_simplification_pass` 尚不能将移位后与掩码的组合模式进一步优化为单一的 `bit_slice`；因此，在 DSLX 层面直接书写 `x[8:16]`（或等价宽度切片）仍然是保证最优 IR 生成的前提。这也从侧面说明了高级语言原生位提取语法对于硬件综合工具链的重要性：它绕过了 C 语言语义鸿沟，使编译器能够直接获取开发者的位级意图。

2.3 Token 机制与隐式同步

在传统数据流 IR 中，由于节点之间仅存在数据依赖边，编译器默认可以任意重排无依赖的运算。然而，当引入带有副作用的操作（如 I/O 发送/接收、断言、状态读写）时，必须提供一种在数据流图中”注入顺序约束”的机制。XLS 的解决方案是引入 **Token** 类型。

Token 在 XLS IR 中是一个零比特宽（zero-width）的特殊值，它不携带任何数据负载，仅在图中传递”序依赖”（ordering dependency）。DSLX 中的 I/O 原语，如 `recv` 和 `send`，都接

受一个 Token 输入并返回一个新的 Token 输出。通过 Token 链，程序员可以精确控制副作用操作的执行顺序。例如，代码清单 3 展示了如何利用 `join` 操作合并来自两条不同通道接收操作的 Token，并在统一的序约束下执行发送操作。

```
proc adder {
    A: chan<u32> in;
    B: chan<u32> in;
    C: chan<u32> out;

    config(A: chan<u32> in, B: chan<u32> in, C: chan<u32> out) {
        (A, B, C)
    }
    init { () }
    next(st: ()) {
        let (tok_A, data_A) = recv(join(), A);
        let (tok_B, data_B) = recv(join(), B);
        let sum = data_A + data_B;
        let tok = join(tok_A, tok_B);
        send(tok, C, sum);
    }
}
```

Listing 3: DSLX 中通过 Token 合并控制收发顺序

如代码所示，`join` 函数将多个 Token 合并为一个，确保只有当来自通道 `A` 和 `B` 的接收操作均完成后，`send` 操作才会被允许执行。这种 Token 驱动的同步在最终生成的 RTL 中不会对应任何实际硬件——Token 在代码生成阶段被完全消除——但其施加的序约束已被调度和状态机生成阶段完整吸收，从而保证了硬件行为与软件仿真严格一致。

2.4 Proc 与 Channel：状态 ful 通信的计算单元

DSLX 中的函数（`fn`）仅用于描述纯组合逻辑或无状态流水线模块。为了表达带状态、支持时序递推的硬件模块，XLS 引入了 **proc**（communicating sequential process）抽象。每个 proc 包含三个固定部分：`config` 用于模块实例化时的参数与通道绑定；`init` 定义状态寄存器的初始值；`next` 接受当前状态并返回下一个状态 [6]。

Proc 之间通过 **channel** 进行通信。抽象语义上，channel 是无限深度的 FIFO 队列：接收操作默认为**阻塞式**（blocking），若上游无数据则当前激活挂起；发送操作在抽象语义上为非阻塞式。这种 timing-insensitive 的设计极大地解放了程序员——他们无需关心时钟周期数或手动插入流水线寄存器，只需声明” 当数据 A 和 B 到齐后执行计算并输出 C”。编译器的职责则是在这一时间无关模型上” 添加时间”，通过调度算法合理插入寄存器并生成握手逻辑。

2.4.1 Proc 架构设计哲学：分离关注点与静态可分析性

XLS 的 proc 设计体现了**分离关注点**（Separation of Concerns）与**完全静态可分析**的核心架构哲学。三个组成部分各司其职：

- **config**：在**结构层**（structural level）执行，负责接口绑定与通道连接，仅在编译时执行一次；
- **init**：定义状态寄存器的初始值，确定复位后的起始状态；
- **next**：在**行为层**（behavioral level）执行，描述每个时钟周期的状态转移与计算逻辑。

config 的独立设计具有深刻的工程考量。首先，它实现了**依赖注入**（Dependency Injection）模式——proc 声明自身需要哪些通道（A: chan<u32> in），而 config 参数决定这些通道由谁提供。这使得同一 proc 可以在不同系统中被复用，只需在 config 中绑定不同的上游/下游通道。

其次，config 支持**嵌套实例化与通道连接**。在复杂系统中，顶层 proc 通过 spawn 实例化子 proc，并在 config 中创建内部通道完成连接：

```
pub proc system {
    external_in: chan<u32> in;
    external_out: chan<u32> out;

    config(external_in: chan<u32> in, external_out: chan<u32> out) {
        // 创建内部通道（仅在此作用域可见）
        let (internal_s, internal_r) = chan<u32>("internal_pipe");

        // 实例化子proc，将外部接口与内部通道绑定
        spawn adder(external_in, internal_r, external_out);

        (external_in, external_out)
    }
    // ...
}
```

Listing 4: 嵌套实例化与通道连接

第三，config 保证了**编译期可分析的确定性**。所有通道连接在编译时即可完全确定，形成静态的数据流图拓扑。这使得 XLS 编译器能够：

1. 静态检测死锁（通过分析通道依赖环）；
2. 精确推断缓冲区深度需求；
3. 生成确定的 RTL 层次结构，无需动态路由或运行时仲裁。

这种设计与 Kahn 进程网络的语义形成严格对应：config 定义了 **Network** 层（连接拓扑），next 定义了 **Process** 层（计算逻辑）。两者的分离使编译器可以分别优化——在 Network 层优化通道深度与仲裁策略，在 Process 层优化流水线调度与状态机生成。

2.5 纯函数与流水线设计

代码清单 5 展示了最基础的纯函数描述。DSLX 函数 add_three 描述了三数相加的组合逻辑。虽然代码中无显式时序概念，但编译器可通过--pipeline_stages 参数将其映射为流水线硬件。

```
fn add_three(a: u32, b: u32, c: u32) -> u32 {
    let sum1 = a + b;
    let sum2 = sum1 + c;
    sum2
}

#[test]
fn test_add_three() {
    assert_eq(add_three(u32:1, u32:2, u32:3), u32:6);
}
```

Listing 5: 简单流水线函数示例

通过 `interpreter_main` 验证测试后，使用 `ir_converter_main` 转换为 IR。生成的 IR 如代码清单 6 所示，采用 Sea-of-Nodes 表示，每个操作是显式依赖图中的节点。

```
package simple_pipeline

top fn __simple_pipeline__add_three(a: bits[32], b: bits[32], c: bits[32])
    -> bits[32] {
  sum1: bits[32] = add(a, b)
  ret sum2: bits[32] = add(sum1, c)
}
```

Listing 6: 生成的 XLS IR 表示

使用 `codegen_main --pipeline_stages=2` 生成双周期流水线 Verilog，如代码清单 7 所示。可见 XLS 自动插入了输入输出寄存器（`*_flop`）和中间流水线寄存器（`p0_*`），实现了时序逻辑与组合逻辑的完整映射。

```
module __simple_pipeline__add_three(
  input wire clk,
  input wire [31:0] a, b, c,
  output wire [31:0] out
);
  // Pipe stage 0: 输入寄存器
  reg [31:0] a__input_flop, b__input_flop, c__input_flop;
  always_ff @(posedge clk) begin
    a__input_flop <= a;
    b__input_flop <= b;
    c__input_flop <= c;
  end

  // Pipe stage 1: 第一级加法
  wire [31:0] p1_sum1_comb = a__input_flop + b__input_flop;
  reg [31:0] p0_c, p0_sum1;
  always_ff @(posedge clk) begin
```

```
    p0_c <= c__input_flop;
    p0_sum1 <= p1_sum1_comb;
  end

  // Pipe stage 2: 第二级加法与输出
  wire [31:0] p2_sum2_comb = p0_sum1 + p0_c;
  reg [31:0] out__output_flop;
  always_ff @(posedge clk) begin
    out__output_flop <= p2_sum2_comb;
  end
  assign out = out__output_flop;
endmodule
```

Listing 7: 生成的流水线 Verilog 代码

2.6 Proc 与状态机实现

代码清单 8 展示了使用 proc 实现有限状态机（FSM）的示例。proc 包含三个核心部分：`config` 定义接口通道绑定，`init` 定义状态寄存器初始值，`next` 描述状态转移逻辑。

```
enum State : u2 { IDLE = 0, RECEIVE = 1, COMPUTE = 2, SEND = 3 }

pub proc fsm_example {
    input_r: chan<u32> in;   // 输入FIFO通道
    output_s: chan<u32> out; // 输出FIFO通道

    config(input_r: chan<u32> in, output_s: chan<u32> out) {
        (input_r, output_s)
    }

    init { State::IDLE }  // 初始状态

    next(state: State) {
        let tok = join();
        match state {
            State::IDLE => {
                let (tok, data, valid) = recv_non_blocking(
                    tok, input_r, u32:0);
                if valid { State::RECEIVE } else { State::IDLE }
            },
            State::RECEIVE => State::COMPUTE,
            State::COMPUTE => State::SEND,
            State::SEND => {
                let tok = send(tok, output_s, u32:42);
                State::IDLE
            },
```

```
            _ => State::IDLE,
        }
    }
}
```

Listing 8: 有限状态机的 DSLX 实现

生成的 IR（代码清单 9）展示了状态机的底层实现。状态变量`__state` 为 2 比特位宽，通过 `state_read` 和 `next_value` 操作实现状态寄存器读写。`priority_sel` 实现状态译码，`send/receive` 操作映射为 FIFO 接口。

```
top proc __state_machine__fsm_example_0_next(__state: bits[2], init={0}) {
  __state: bits[2] = state_read(state_element=__state)
  // 状态译码
  eq.6: bits[1] = eq(literal.5, __state)  // IDLE?
  eq.25: bits[1] = eq(literal.24, __state) // RECEIVE?
  // ...
  // 非阻塞接收
  receive.10: (token, bits[32], bits[1]) = receive(
      tok, predicate=and.9,
      channel=state_machine__input_r, blocking=false)
  // 下一状态选择
  priority_sel.46: bits[2] = priority_sel(concat.45,
      cases=[sel.23, literal.28, literal.33, literal.42])
  next_value.47: () = next_value(param=__state, value=priority_sel.46)
}
```

Listing 9: 状态机的 IR 表示（节选）

2.7 Token 机制与精确时序控制

代码清单 10 展示了多接口时序控制场景。通过 Token 链确保操作顺序：先读取控制信号，再依次读取数据 A 和 B（形成串行依赖），最后先发送结果再发送状态报告。

```
pub proc interface_timing {
    data_a_r: chan<u32> in;
    data_b_r: chan<u32> in;
    ctrl_r: chan<u8> in;
    result_s: chan<u32> out;
    status_s: chan<u8> out;

    next(state: ()) {
        let base_tok = join();
        // 步骤1: 接收控制信号
        let (ctrl_tok, ctrl) = recv(base_tok, ctrl_r);
        // 步骤2: 接收数据A (依赖控制信号)
        let (data_a_tok, data_a) = recv(ctrl_tok, data_a_r);
```

```
        // 步骤3: 接收数据B (依赖数据A)
        let (data_b_tok, data_b) = recv(data_a_tok, data_b_r);
        // 计算
        let result = match ctrl {
            u8:0 => data_a + data_b,
            u8:1 => data_a - data_b,
            _ => data_a | data_b,
        };
        // 步骤4: 合并Token
        let compute_tok = join(data_b_tok);
        // 步骤5: 发送结果
        let result_tok = send(compute_tok, result_s, result);
        // 步骤6: 发送状态 (依赖结果发送)
        let final_tok = send(result_tok, status_s, ctrl + u8:1);
    }
}
```

Listing 10: Token 驱动的接口时序控制

代码清单 11 展示了带优先级仲裁的接口设计。使用 `recv_non_blocking` 实现时序敏感操作，高优先级数据可抢占低优先级通道。

```
pub proc priority_arbiter {
    high_priority_r: chan<u32> in;
    low_priority_r: chan<u32> in;
    output_s: chan<(u1, u32)> out;

    next(state: ()) {
        let tok = join();
        // 非阻塞检查高优先级
        let (tok_hp, hp_data, hp_valid) =
            recv_non_blocking(tok, high_priority_r, u32:0);
        // 仅当无高优先级时检查低优先级
        let (tok_lp, lp_data, lp_valid) =
            recv_if_non_blocking(tok, low_priority_r, !hp_valid, u32:0);
        // 仲裁决策
        let (source, data, valid) = if hp_valid {
            (u1:0, hp_data, true)
        } else if lp_valid {
            (u1:1, lp_data, true)
        } else { (u1:0, u32:0, false) };
        // 条件发送
        let final_tok = send_if(arb_tok, output_s, valid, (source, data));
    }
}
```

Listing 11: 优先级仲裁器实现

2.8 多 Proc 协作系统

代码清单 12 展示了由多个 proc 组成的复杂数据处理系统。data_generator 产生带校验和的数据包，data_processor 根据 header 类型执行不同运算，data_collector 收集并统计结果。各 proc 通过 FIFO 通道连接，形成完整的数据通路。

```
struct DataPacket {
    header: u8,       // 数据类型标识
    payload: u32,     // 实际数据
    checksum: u8,     // 校验和
}

pub proc data_generator {
    output_s: chan<DataPacket> out;
    config_s: chan<u32> in;

    next(state: (u32, u32)) {
        let (count, target) = state;
        let tok = join();
        // 生成带校验和的数据包
        let next_count = if count < target {
            let header = (count as u8) & u8:0x03;
            let payload = count * u32:7;
            let checksum = (header + (payload as u8)) & u8:0xFF;
            let packet = DataPacket { header, payload, checksum };
            let tok = send(tok, output_s, packet);
            count + u32:1
        } else { count };
        (next_count, target)
    }
}

pub proc data_processor {
    input_r: chan<DataPacket> in;
    output_s: chan<u32> out;

    next(state: ()) {
        let tok = join();
        let (recv_tok, packet) = recv(tok, input_r);
        // 根据header处理数据
        let result = match packet.header {
            u8:0 => packet.payload + u32:1,
            u8:1 => packet.payload * u32:2,
            u8:2 => packet.payload ^ u32:0xFFFF,
            u8:3 => packet.payload >> u32:2,
            _ => packet.payload,
```

```
        };
        // 校验和验证
        let expected = (packet.header + (packet.payload as u8)) & u8:0xFF;
        let valid = packet.checksum == expected;
        let send_tok = send_if(recv_tok, output_s, valid, result);
    }
}
```

Listing 12: 多 Proc 协作系统架构

2.9 部署与开发流程

XLS 工具链的主要组件包括：`interpreter_main`（DSLX 解释器与测试运行器）、`ir_converter_main`（DSLX 到 IR 转换器）、`opt_main`（IR 优化器）、`codegen_main`（Verilog 生成器）。典型开发流程为：

1. **编写 DSLX 代码**：定义函数或 proc，添加`#[test]` 或`#[test_proc]` 测试；
2. **本地仿真验证**：`interpreter_main demo.x` 运行测试；
3. **转换为 IR**：`ir_converter_main --top=<name> demo.x > demo.ir`；
4. **优化 IR**：`opt_main demo.ir > demo.opt.ir`；
5. **生成 Verilog**：`codegen_main --generator=pipeline --pipeline_stages=N demo.opt.ir`。

实践表明，XLS 编译器能自动处理流水线寄存器插入、状态机生成、握手信号（valid/ready）等底层细节，开发者仅需关注数据流逻辑。但需注意：proc 生成流水线代码需要更复杂的调度配置，当前版本对带复位信号的状态机支持仍在完善中。

3 XLS 的理论定位：KPN 的当代实践

通过将 XLS 置于 KPN 和 Actor 模型的理论谱系中考察，可以更深刻地理解其设计选择。XLS 并非简单地”采用”了 KPN，而是在继承 KPN 核心思想的基础上，针对硬件综合的特殊需求进行了关键性的扩展和改进。

3.1 XLS 与 KPN 理论的直接对应

XLS 的 proc/channel 模型与 Kahn 1974 年原始论文中定义的进程网络存在严格的结构对应关系。表 2 展示了这种映射：

XLS 的 `proc` 结构可以看作是 Kahn 进程在硬件约束下的精化。Kahn 原始论文将进程定义为”从输入历史到输出历史的连续映射”，这一纯数学视角在 XLS 中被具体化为：`next` 函数接受当前状态和输入 token，产生下一状态和输出 token。这种精化保留了 KPN 的确定性保证——给定相同的输入序列，XLS proc 总是产生相同的状态转移序列和输出序列。

表 2: KPN 理论与 XLS 实现的对应关系

KPN 概念	Kahn 1974 定义	XLS 实现
进程 (Process)	连续函数 $P : S^\omega \to S^\omega$	`proc` 的 `next` 函数
通道 (Channel)	无界 FIFO 队列	`chan<T>`，综合时确定深度
读操作	阻塞式破坏性读取 (`wait`)	`recv`，阻塞语义
写操作	非阻塞式发送	`send`，带反压机制
确定性	相同输入历史必产生相同输出	软件仿真与 RTL 严格等价
连续性	有限输出依赖有限输入	状态机推进的有限步计算

3.2 proc 的形式化语义：token 驱动的 Mealy 机

若从有限状态机的经典分类审视，DSLX 的 `proc` 完全符合 **Mealy 机**的定义：

$$\mathsf{next} : (\mathsf{State}, \mathsf{Inputs}) \longrightarrow (\mathsf{NextState}, \mathsf{Outputs})$$

当前状态 ＋ 当前输入决定**下一状态**和**输出**，这正是 Mealy 机（而非 Moore 机）的特征。以 MAC PE 为例：`state=(acc, cycle)`，`inputs=(weight, activ)`（通过 `recv` 从 channel 读取），`outputs=send(tok, out_s, new_acc)`，`next_state=(new_acc, next_cycle)`。

然而，XLS 的 proc 不能简单等同于传统硬件 Mealy 机。传统 Mealy 机强约束于时钟：每个周期状态必须更新一次，输入/输出严格绑定在时钟沿上。而 XLS 通过 token 机制获得了三个关键自由度：

1. **自步进（Self-timed）**：`recv` 可以阻塞。若上游数据未就绪，proc 在抽象语义上”挂起”，不消耗任何逻辑周期。编译器生成的 FSM 只是在当前状态等待，但开发者无需显式描述这一等待逻辑。

2. **输出的条件性与序列化**：token 链允许将多步顺序 I/O（如先 `send(result)` 再 `send(status)`）编码在同一个 `next` 函数体内，而编译器负责决定这些逻辑步骤被映射到同一个物理周期还是多个周期。

3. **输入的组合与分解**：`join()` 操作将多个并行输入线聚合为一个显式同步点，这比传统电路”所有输入同时到达”的假设更灵活——proc 可以选择性地只读取部分输入，或强制多个输入在某一时刻同步。

Lowering 的映射本质因此可以概括为：XLS 编译器将一个 **token 驱动的 Mealy 机**安全地转换为一个**时钟驱动的同步 Mealy 机**。调度器决定哪些 `recv`/`send` 可以在同一周期完成，状态寄存器何时更新，以及阻塞等待如何映射为 `valid`/`ready` 握手的保持状态。通过单一 IR、解释器和 IR Fuzzer 的三重验证，XLS 保证：无论调度器如何压缩或展开周期，生成的 RTL 与原始 proc 在可观察行为上严格等价。

3.3 从数据流 Actor 到 Kahn Process 的转换

Tretter 等人在 2015 年的工作提出了将 Dataflow Actor 自动转换为 Kahn Process 的算法 [8]，这一研究为理解 XLS 的编译流程提供了重要视角。在 CAL Actor Language 等多媒体处理语言

中，actor 可以通过 `guard` 条件基于输入数据的可用性和内容进行选择性触发，这带来了非确定性的可能。Tretter 等人的分类算法可以识别出那些实际上是 KPN 兼容的 actor——即其行为不依赖于数据到达的时序。

XLS 的 DSLX 语言设计采取了更严格的立场：它直接要求所有 `proc` 必须是 KPN 兼容的。`recv_non_blocking` 虽然允许检查数据可用性，但 XLS 的调度器确保这种检查不会引入时序相关的非确定性。这种设计选择牺牲了一定的表达灵活性（无法直接实现非确定性 merge），但换取了：

1. **可静态分析的确定性**：编译器可以在综合前验证设计的确定性；
2. **更高效的实现**：无需运行时调度器评估 guard 条件，所有控制流在编译时即已确定；
3. **软硬件语义等价**：软件仿真行为与 RTL 实现严格一致。

3.4 XLS 对 KPN 的扩展

尽管 XLS 的核心基于 KPN，但它针对硬件综合的具体需求引入了 Kahn 原始理论中不存在的关键扩展：

（1）Token 机制与显式时序控制。Kahn 的原始模型完全抽象了时间概念——进程仅在输入可用时计算，不关心计算需要多长时间。然而，硬件是时间敏感的：寄存器在每个时钟沿采样，信号传播有延迟约束。XLS 的 Token 机制在保持 KPN 确定性语义的同时，提供了”注入时序约束”的能力。`join` 操作强制多个数据依赖的同步点，`send`/`recv` 的 Token 链确保 I/O 操作按指定顺序执行。这些 Token 在最终的 RTL 中完全消失，但它们施加的序约束已被调度器吸收。

（2）Config 阶段的显式化。KPN 理论假设进程网络的结构是静态给定的。XLS 通过 `config` 阶段显式化了这一结构绑定过程，使其成为可编程、可复用的抽象。开发者可以在 `config` 中参数化地实例化子进程、创建内部通道、建立连接拓扑。这与 Kahn 和 MacQueen 1977 年论文中讨论的协程网络有相似之处，但 XLS 将其提升为一等语言特性。

（3）有限缓冲与反压。Kahn 原始模型的”无界 FIFO”假设在物理硬件中不可能实现。XLS 通过引入阻塞式 `recv` 和带反压的 `send` 解决了这一问题。当下游缓冲区满时，`send` 操作会阻塞（在抽象语义层面），这一反压信号会向上游传播，形成流水线 stall。这种设计保持了 KPN 的确定性——stall 只是时序上的延迟，不影响最终计算结果——同时使系统可实际综合。

3.5 为什么选择 KPN 而非 Actor 模型

在并发计算的两大理论支柱之间，XLS 明确选择了 KPN 而非 Actor 模型。这一选择反映了硬件综合领域的特殊约束：

确定性需求。硬件设计通常要求确定性行为——相同的输入激励必须产生相同的输出响应。KPN 的确定性保证使其更适合作为硬件综合的源语言语义。Actor 模型的非确定性（消息到达顺序影响行为）虽然在分布式系统中是自然且有用的，但会给硬件验证带来复杂性。

静态拓扑。硬件的物理连接在制造时即已固定，无法像软件 Actor 那样动态创建新的通信链路。KPN 的静态网络拓扑与这一物理现实直接对应。Actor 模型的动态拓扑（可以动态创建 Actor 和传递地址）虽然灵活，但在硬件中实现需要复杂的运行时支持。

同步与异步的折中。KPN 采用”异步进程+同步 rendezvous”的混合模型——发送非阻塞但接收阻塞，这恰好对应硬件 FIFO 的接口语义（ready/valid 握手）。Actor 模型的纯异步消息传递（消息入 mailbox 即可返回）在硬件中实现需要更深的缓冲区和更复杂的流控。

然而，XLS 的设计也借鉴了 Actor 模型的某些思想。`proc` 封装私有状态、只能通过 channel 通信的设计，与 Actor 的封装性和消息传递哲学是一致的。可以说，XLS 的 proc 是”受限的 Actor”——它放弃了动态创建和地址传递的灵活性，换取了静态可分析性和确定性保证。

3.6 历史脉络：从 Manchester 到 XLS

理解 XLS 的历史定位，需要回顾数据流计算机架构的发展历程。Gurd 等人 1985 年发表的 Manchester 原型数据流计算机 [9] 是这一领域的重要里程碑。该机器采用”标记令牌”（Tagged-Token）模型，通过动态标记支持代码的重入执行和递归调用，实现了细粒度的数据流并行。

Manchester 机器的经验教训深刻影响了后来的数据流系统：

- **细粒度同步开销**：令牌匹配和标记管理的开销抵消了并行带来的好处；
- **局部性利用不足**：数据在全局网络中流动，难以利用局部缓存；
- **控制结构困难**：高效实现循环和条件分支需要复杂的硬件支持。

XLS 通过以下设计回应了这些挑战：

- **粗粒度数据流**：以 proc 为单位而非单条指令，摊销同步开销；
- **编译时调度**：通过 SDC/Min-Cut 调度器静态确定执行顺序，消除运行时调度的不确定性；
- **层次化存储**：支持数组、结构体等复合类型，允许开发者显式管理数据局部性。

从 Manchester 的硬件标记令牌到 XLS 的编译时 Token 分析，数据流计算模型经历了一次重要的范式转移：从”运行时数据驱动”转向”编译时数据流分析”。这一转移牺牲了一定的动态灵活性，但换取了可预测的性能和高效的硬件实现。

4 XLS IR 与编译优化

4.1 单一 Sea-of-Nodes 中间表示

XLS 最显著的设计决策之一，是在整个编译流程中采用**单一的中间表示（IR）**，从前端（DSLX 或 xlscc）一直贯穿到 RTL 生成阶段 [6]。这与 LLVM 或 MLIR-CIRCT 中”多 dialect、多层 IR 转换”的策略形成鲜明对比。XLS 团队认为，由于硬件本身就是数据流结构，从前端的数据流表示平滑下降到底层 RTL 数据流表示，specialization 与 reusability 之间的权衡并不必要。

XLS IR 显式**不是基于控制流图（CFG）**的。相反，它采用 **Sea-of-Nodes（SoN）**表示：每个操作是一个图中的节点，操作数引用其前驱节点，整个图构成有向无环图（DAG）。由于 IR 是函数式的，SSA（Static Single Assignment）属性自动成立，无需显式的 ϕ 节点插入与维护。DSLX 中的条件表达式（`if/else`）在 lowering 过程中被转换为 `Select`（二进制编码选择器）或 `OneHotSelect`（独热编码选择器）节点。`OneHotSelect` 通常比 `Select` 具有更低的组合逻辑延迟，因为后者等价于”译码+OneHotSelect”的复合操作。

4.2 IR 优化流程

XLS 的优化流程通过一系列 function-level 和 proc-level 的 pass 对 IR 进行转换。根据自动生成的 pass 文档和源码分析 [6]，核心优化 pass 可分为以下几类：

(1) 传统编译器优化。包括死代码消除（DCE）、公共子表达式消除（CSE）和常量折叠。在 SoN 表示中，CSE 的实现尤为简洁：若两个节点 hash 值与操作数完全相同，只需将后者的所有使用边重定向到前者，随后由 DCE 清理即可。常量折叠直接复用 XLS 解释器对仅含 Literal 操作数的节点进行求值，确保折叠结果与目标硬件语义严格一致。

(2) 位级窄化优化 (Narrowing)。这是 XLS 最具特色的硬件专属优化之一。编译器执行位流分析（bitwise flow analysis），推断出某些运算输出中的特定位恒为 0 或恒为 1（known bits）。基于这一信息，`narrowing_pass` 可以缩减运算的位宽，从而降低组合逻辑延迟与面积。例如，若一个 OneHotSelect 的高两位和低两位在所有输入臂中都是常量，该操作可以被" 挤压" 到仅保留变化的中间位。

(3) Select 优化。DSLX 中常见的嵌套条件表达式（等价于 C 语言中的嵌套三元运算符）在 lowering 后可能形成线性 `Select` 链。`select_simplification_pass` 和 `select_merging_pass` 可以将这种链式结构转换为单个 `OneHotSelect`，把串行依赖转化为并行选择，显著降低关键路径延迟。此外，范围分析（range analysis）还可用于" 稀疏化" 选择器，剔除不可能到达的选择臂。

(4) 算术强度削弱与重结合。`arith_simplification_pass` 将除以常数的操作替换为"magic multiplication"（移位+乘法序列），将乘以小常数替换为移位加减组合，并对加法链进行重结合以形成平衡树，降低关键路径深度。

4.3 算术简化 Pass 的位级优化实现

为进一步理解 XLS 的 IR 级代数优化能力，本节基于对 Google XLS 主线源码（`xls/passes/arith_simplification_pass.cc`）的系统分析 [4]，对 `arith_simplification_pass` 支持的算术化简规则进行详细梳理。该 Pass 以 fixed-point 方式迭代运行，直到 IR 图不再发生变化为止。表 3 归纳了其涵盖的十类主要优化。

在众多规则中，移位相关优化与 `bit_slice` 的交互尤为值得注意。该 Pass 能够将常数逻辑移位直接重写为 `Concat` 与 `BitSlice` 的组合，从而避免不必要地实例化 barrel shifter。代码清单 13 展示了这一变换的 IR 形态。

```
// 原始 IR（左侧）：逻辑右移 8 位
shrl(x: bits[32], 8)

// 优化后 IR（由 arith_simplification_pass 生成）：
// Concat(UBits(0, 8), BitSlice(x, start=8, width=24))
```

Listing 13: 常量逻辑移位优化为 `Concat+BitSlice` 的示例

算术右移（`shra`）的常数情形则被重写为 `sign_extend(BitSlice(...))`。这一系列变换体现了 XLS 在 IR 层级对位级操作的精细控制。然而，基于对 `arith_simplification_pass.cc` 源码的逐行分析，可以确认一个明确的事实：**该 Pass 完全不包含对 `Op::kAnd`（按位与）的处理规则**。这意味着在实际硬件设计中极为常见的" 位提取方言"——例如 `(x >> c) & mask` 或 `(x << c) & mask`——**不会被该 Pass 进一步折叠为单一的 `BitSlice` 操作**。

表 3: XLS `arith_simplification_pass` 支持的算术优化分类

优化类别	**具体规则**
除法/取模强度削弱	除以 2^k → `shrl`；模 2^k → `BitSlice`+zero-extend；除以非 2 幂常量（无/有符号）→ magic multiplication/`MULSH` 序列；小常量被除数 → LUT 查表
乘法强度削弱	乘 2^k → `shll`；乘小正数常量 → PopCount 驱动的移位加减树；乘法结果仅被窄 `slice` 使用时 → 将窄化折叠进乘法节点
移位优化	常量逻辑左移/右移 → `Concat(BitSlice, 0)`；常量算术右移 → `sign_extend(BitSlice)`；1 位值移位特化；移除移位的 zero-ext 保护
Decode 优化	1 位宽 `decode` → `eq(index, 0)`；`decode(N)-1`（即 $(1 \ll N) - 1$）→ `!(all_ones` $\ll$ `N)`
加减法/取反	`Add(Not(x), 1)` → `Neg(x)`；双重 sign-extend 合并；`add(sign_ext(u1), K)` → select
比较优化	`eq(-lhs,-rhs)` 去负号；单射运算比较（如 $X + C_0 == C_1$）→ $X == C_1 - C_0$；与 LSB 连续 1 的 mask 比较 → reduce 逻辑
Select 模式	`sel(pred,[x,neg(x)])==0` → `x==0`（abs 逻辑）
扩展/窄化	`NarrowOrExtend` 辅助；常量除法中的预截断与后扩展
1 位值特化	多位移位的 corner-case 化简
迭代精化	fixed-point 反复应用，直至收敛

具体而言，当输入代码为 `(x >> 8) & 0xFF` 时，`arith_simplification_pass` 只会先将 `x >> 8` 展开为 `Concat(UBits(0,8), BitSlice(x,8,24))`，但随后与 `0xFF` 的按位与运算会保留下来，而不会被化简为干净的 `BitSlice(x, 8, 8)`。虽然相邻的 `bit_slice_simplification_pass` 能够将某些 `BitSlice` 穿过移位操作进行提升（例如 `BitSlice(Shll(x,y))` → `Shll(BitSlice(x),y)`），但它同样不处理 `And` 与移位的联合模式。这构成了 XLS 当前优化管线中的一个可改进空间。

4.4 形式化验证与 Fuzzing

XLS 在工具链中深度集成了基于 Z3 SMT 求解器的形式化验证和专用 IR Fuzzer[4]。Fuzzer 通过生成随机 IR 并与解释器/JIT 的执行结果进行交叉验证，持续检测优化 pass 或代码生成器可能引入的语义偏差。这种”软件风格”的测试方法论，对于保证软硬件语义等价至关重要。

4.5 Proc lowering 与 Token 消除机制

为进一步理解 proc 的 lowering 机制，本节展示一个经过实际编译验证的 MAC Engine（乘累加 PE 单元）示例，及其从 DSLX 到 IR 再到 Verilog 的完整映射过程。

```
pub proc mac_pe {
    weight_r: chan<u32> in;
    activ_r: chan<u32> in;
    out_s: chan<u32> out;

    config(weight_r: chan<u32> in, activ_r: chan<u32> in,
           out_s: chan<u32> out) {
        (weight_r, activ_r, out_s)
    }

    init { (u32:0, u1:0) }  // (acc, cycle)

    next(state: (u32, u1)) {
        let tok = join();
        let (tok, weight) = recv(tok, weight_r);
        let (tok, activ)  = recv(tok, activ_r);

        let (acc, cycle) = state;
        let new_acc = acc + weight * activ;
        let next_cycle = cycle ^ u1:1;

        let tok = if next_cycle == u1:0 {
            send(tok, out_s, new_acc)
        } else { tok };

        (new_acc, next_cycle)
    }
```

```
}
```

Listing 14: MAC PE 的 DSLX 实现

在 lowering 到 IR 时，编译器生成了如代码清单 15 所示的 Sea-of-Nodes 表示。IR 中的每个 `receive` 和 `send` 都显式体现为节点，而 token 的传递路径（`tok` → `tok__1` → `tok__2`）编码了严格的时序顺序。`state_read` 节点读出了 proc 的状态寄存器，`next_value` 节点负责在周期结束时写回新状态。这揭示了 proc lowering 的核心本质：**token 链是隐式的 FSM 状态转移图，而 `state_read/next_value` 则是显式的状态寄存器读写操作**。

```
top proc __mac_pe__mac_pe_0_next(__state: (bits[32], bits[1]),
                                 init={(0, 0)}) {
  tok: token = after_all(id=4)
  receive.5: (token, bits[32]) = receive(tok, channel=mac_pe__weight_r, id
      =5)
  tok__1: token = tuple_index(receive.5, index=0, id=7)
  receive.9: (token, bits[32]) =
      receive(tok__1, channel=mac_pe__activ_r, id=9)
  weight: bits[32] = tuple_index(receive.5, index=1, id=8)
  activ: bits[32] = tuple_index(receive.9, index=1, id=12)
  umul.15: bits[32] = umul(weight, activ, id=15)
  __state: (bits[32], bits[1]) = state_read(state_element=__state, id=2)
  acc: bits[32] = tuple_index(__state, index=0, id=13)
  new_acc: bits[32] = add(acc, umul.15, id=16)
  cycle: bits[1] = tuple_index(__state, index=1, id=14)
  next_cycle: bits[1] = xor(cycle, literal.17=1, id=18)
  // 条件发送与next_value更新状态...
}
```

Listing 15: mac_pe lowering 后的 IR 片段

使用 `codegen_main` 生成 proc 的 Verilog 时，当前预编译二进制可能触发一个已知内部错误（对应 GitHub issue #743）——“Register has a reset value but corresponding register write operation has no reset operand”。其根本原因是：proc 的 `init` 会被 lowering 为带复位初值的寄存器，但 `codegen_main` 默认不生成复位端口。**解决方案**：为 `codegen_main` 显式提供复位信号名称：

```
codegen_main --reset=rst --generator=pipeline --pipeline_stages=1 mac_pe.
    opt.ir
```

加此参数后，生成的高质量 Verilog 结构如代码清单 16 所示。

```
module __mac_pe__mac_pe_0_next(
  input wire clk,
  input wire rst,
  input wire [31:0] mac_pe__activ_r,
  input wire mac_pe__activ_r_vld,
  input wire mac_pe__out_s_rdy,
```

```
  input wire [31:0] mac_pe__weight_r,
  input wire mac_pe__weight_r_vld,
  output wire mac_pe__activ_r_rdy,
  output wire [31:0] mac_pe__out_s,
  output wire mac_pe__out_s_vld,
  output wire mac_pe__weight_r_rdy
);
  reg [31:0] ____state_0;  // acc寄存器
  reg ____state_1;         // cycle计数器
  reg [31:0] __mac_pe__activ_r_reg;
  reg __mac_pe__activ_r_valid_reg;
  // ... 输入缓冲寄存器 ...
  wire active_inputs_valid_0 =
      __mac_pe__activ_r_valid_reg & __mac_pe__weight_r_valid_reg;
  assign umul_156 = umul32b_32b_x_32b(__mac_pe__weight_r_reg,
                                      __mac_pe__activ_r_reg);
  assign new_acc = ____state_0 + umul_156;
  assign next_cycle = ~____state_1;
  assign __mac_pe__out_s_vld_buf =
      active_inputs_valid_0 & ____state_1;  // cycle==1时输出
  always_ff @ (posedge clk) begin
    if (rst) begin
      ____state_0 <= 32'h0000_0000;  // init的u32:0
      ____state_1 <= 1'h0;           // init的u1:0
    end else begin
      ____state_0 <= stage_outputs_valid_0 ? new_acc : ____state_0;
      ____state_1 <= stage_outputs_valid_0 ? next_cycle : ____state_1;
    end
  end
endmodule
```

Listing 16: MAC PE 生成的流水线 Verilog

生成的 RTL 并未直接使用显式的 4 状态 FSM，而是采用了**数据流流水线化的缓冲结构**：通过 `valid_reg` 追踪每个输入是否就绪，用 `active_inputs_valid_0` 聚合前置条件，再用 `stage_outputs_valid_0` 控制状态推进和输出使能。这说明 XLS 编译器有能力将 token 链语义优化为更高效的并行缓冲实现，而不仅仅是朴素的状态机翻译。

4.6 Pipeline 调度

调度（Scheduling）是 XLS 将”无时间”数据流模型映射到”同步时钟驱动 RTL”的核心步骤。XLS 的 pipeline scheduling 任务可描述为：在满足目标时钟周期的组合逻辑延迟约束前提下，将 DAG 中的节点分配到离散的 pipeline stage，并最小化 stage 间所需寄存器的数量。

如算法 1 所示，调度过程分为两个阶段。第一阶段确定**有效时钟周期**（effective clock period）：若用户显式指定了时钟周期，则直接采用；若仅指定了 stage 数量，编译器则通过二分搜索找出

满足该 stage 数的最小时钟周期。第二阶段在给定时钟周期约束下，求解各节点的 stage 分配，以最小化寄存器总数。

算法 1 XLS Pipeline Scheduling 流程

输入： 数据流图 $G = (V, E)$，延迟模型 $\mathcal{D}$，可选周期 T_{target}，可选 stage 数 S_{target}

输出： 每个节点 $v \in V$ 的 stage 分配 $s(v)$，总寄存器数最小化

```
if T_target 已指定 then
    T_eff ← T_target × (1 − margin)                    ▷ 应用时钟裕量
else
    通过二分搜索求解满足 ∃ 可行 S_target-stage 调度的最小 T
    T_eff ← T × (1 + relaxation)                       ▷ 可选松弛
end if
使用 SDC 或 Min-Cut 算法求解满足 D 约束的 stage 分配
目标：最小化 Σ_{v∈V} width(v) × (max_{u∈users(v)} s(u) − s(v))
return stage 分配方案
```

XLS 实现了两种调度后端。**SDC Scheduler** 将调度问题建模为差分约束系统（System of Difference Constraints），调用 Google OR-Tools 的数学规划求解器进行求解 [4]。**Min-Cut Scheduler** 则采用基于最小代价图割的迭代划分策略：通过反复将可跨越 stage 边界的节点集合以最小代价割开，收紧节点的上下界直至所有节点被分配到唯一 stage[4]。这两种算法分别以数学精确性和工程实现灵活性见长，用户可根据设计特点选择。

4.7 RTL Lowering

调度完成后，`codegen_main` 负责将 IR+Schedule 转换为 Verilog 或 SystemVerilog。Lowering 过程中有几个关键操作值得注意：

值展平 (Value Flattening)。DSLX 中的数组（`array`）、元组（`tuple`）和结构体（`struct`）需要展平为 SystemVerilog 中的位向量拼接。例如，4 元素 $u4$ 数组按从最高有效位到最低有效位顺序拼接；元组同样拼接，第 0 个元素位于最高有效位 [6]。

Token 消除。如前所述，Token 在 RTL 中完全不存在，其序约束已通过调度结果和状态机逻辑内化。

Proc 状态机生成。`next` 函数中的状态更新被编译为寄存器的下一个状态逻辑；proc 的激活在硬件中对应每个时钟周期一次的状态推进，当遇到阻塞式接收且 FIFO 为空时，pipeline 会产生反压（backpressure），stall 后续 stage 直到数据可用。

5 黑箱优化哲学与后端融合机制

传统 HLS 工具（如 Vivado HLS）通常依赖人工设计的算法模板（如 Booth 编码、华莱士树）来优化特定算子（如乘法器）。XLS 采用了根本不同的**黑箱**（Black-Box）优化哲学：将算子视为具有延迟-面积特性的抽象原语，通过统一的 IR 表示和自动化调度实现优化，而非依赖人工硬编码的算法结构。

5.1 延迟估计：从结构到特性

XLS 不内置 Booth 编码、华莱士树等具体实现，而是通过**延迟扫描建模**（Delay Characterization）获取操作的物理特性。工具链使用综合工具（如 Yosys、DC、Genus）对从 2 位到 2048 位的各类操作进行预扫描，测量实际延迟并拟合曲线：

$$\text{Delay}(bw) = a \cdot bw + b \cdot \log_2(bw) + c \tag{1}$$

其中 bw 为位宽，系数 (a, b, c) 通过最小二乘法拟合实测数据获得。对于乘法操作，扫描结果隐含了综合工具自动选择的实现方式（无论是 Booth 编码、华莱士树还是工艺库宏单元），XLS 仅需关注"32 位乘法在该工艺下需要多少 ps" 这一抽象特性。

这种设计的优势在于：

- **工艺无关性**：延迟模型可通过更换扫描工艺库（sky130、asap7 等）轻松适配不同制程；
- **自动演进**：当综合工具改进乘法器生成算法时，XLS 自动受益，无需修改编译器代码；
- **统一抽象**：所有操作（加、乘、选择、比较）均采用相同的延迟模型接口，简化调度器设计。

5.2 IR 级优化：结构无关的变换

XLS 的优化 Pass 在 Sea-of-Nodes IR 上操作，将乘法等复杂算子视为**原子节点**，通过结构无关的变换实现优化：

强度削减（Strength Reduction）：将乘以小常数替换为移位和加法序列：

```
// 原始: x * 8
// 优化后: (x << 3)  // 8 = 2^3, 单周期完成
```

重结合（Reassociation）：将线性依赖链转换为平衡树，降低关键路径深度：

```
// 原始: a * b * c * d  // 关键路径: 3个乘法器延迟
// 重结合后: (a * b) * (c * d)  // 关键路径: 2个乘法器延迟
```

位窄化（Narrowing）：基于位级数据流分析，若乘法结果仅使用低 16 位，自动将 32×32 乘法窄化为 16×16。

这些变换不依赖乘法的内部结构（Booth 编码或华莱士树），而是在 IR 图上进行**代数重写**，由后端综合工具决定最终的门级实现。

5.3 调度器：自动流水线打包

XLS 的核心创新在于通过**调度器**（Scheduler）而非人工模板实现时序优化。调度器将数据流图视为带延迟属性的节点集合，使用数学规划方法自动确定流水线切分点：

- **SDC 调度器**：将问题建模为差分约束系统（System of Difference Constraints），调用 Google OR-Tools 求解器求解；
- **Min-Cut 调度器**：基于最小代价图割的迭代划分，逐步收紧节点 stage 边界。

调度器的输入是延迟模型提供的各操作延迟估计，输出是每个节点的 stage 分配方案。这与传统 HLS 形成鲜明对比——后者通常要求开发者显式指定`#pragma HLS pipeline`并手动设计内部循环结构，而 XLS 通过延迟模型**自动决定**操作分组。

5.4 后端接口：行为级 Verilog 作为抽象边界

XLS 不直接生成门级网表（如展开为与门阵列的 Booth 编码），而是生成**行为级 Verilog**：

```
// XLS生成的Verilog
module multiplier(
  input [31:0] a, b,
  output [31:0] result
);
  assign result = a * b;  // 行为级描述
endmodule
```

最终的算法结构决策（是否使用 Booth 编码、华莱士树压缩层次等）被推迟到后端综合阶段，由 Yosys、Design Compiler 等工具根据工艺库和约束条件自动完成。这种分层抽象使 XLS 与后端解耦：XLS 专注于高层数据流优化，后端工具专注于门级映射，两者通过行为级 RTL 作为清晰接口。

5.5 设计范式对比

表 4 对比了 XLS 与传统 HLS 在优化哲学上的差异：

表 4: 传统 HLS 与 XLS 的优化哲学对比

维度	传统 HLS（Vivado/Vitis）	XLS
乘法实现	内置 Booth/华莱士树模板	黑箱延迟模型
优化层次	门级结构优化	IR 级代数变换
流水线划分	人工`#pragma`指导	SDC/Min-Cut 自动调度
后端接口	门级或 RTL 网表	行为级 Verilog
面积-延迟权衡	人工选择模板参数	调度器自动打包

5.6 多周期操作的原子性限制

XLS 的黑箱优化哲学带来了一个根本性限制：**操作的原子性**。在 XLS IR 中，每个节点（如乘法、加法、选择器等）被视为不可分割的原子单元，调度器必须将整个操作分配到单一的 pipeline stage 中。这一设计选择深刻影响了 XLS 处理复杂运算（如大位宽乘法）的能力。

5.6.1 原子操作的语义约束

在 XLS 的 Sea-of-Nodes IR 中，数据依赖边表示”前驱节点的输出作为后继节点的输入”。这一语义隐含了操作的原子性假设——当某个节点被激发 (fire) 时，其所有输入必须已就绪，且该节点的计算必须在当前 stage 内完成。

调度器使用 SDC（System of Difference Constraints）算法求解 stage 分配时，核心约束可表述为：

$$s(v) + \delta(v) \leq s(u) \quad \forall \text{ edge } (v, u) \in E \tag{2}$$

其中 $s(v)$ 表示节点 v 被分配的 stage，$\delta(v)$ 表示操作 v 的延迟（以时钟周期为单位）。由于 $\delta(v) \in \{0, 1\}$，这意味着：

- 若操作延迟小于一个时钟周期（$\delta = 0$），可与后继操作共处同 stage；
- 若操作延迟超过一个时钟周期（$\delta = 1$），必须独占一个 stage；
- **不存在 $\delta > 1$ 的情况**——单个操作无法跨越多个 stage。

5.6.2 大位宽乘法的流水线困境

考虑一个 32 位有符号乘法（$s32 \times s32 \rightarrow s64$）。根据 XLS 的延迟扫描模型，该操作在典型工艺下的延迟可能达到 2-3ns。若目标时钟周期为 1ns（1GHz），则该操作无法在一个 stage 内完成。

在传统 HLS 工具（如 Vivado HLS）中，开发者可以通过`#pragma HLS pipeline`让工具自动将乘法拆分到多周期，或手动实例化多周期 DSP 单元。**在 XLS 中，这两种方式均不可行**。若尝试将高延迟乘法放入流水线，`codegen_main`会产生延迟超标的错误，提示增大周期数或 stage 数。

5.6.3 手动状态机实现方案

当前 XLS 中实现多周期乘法的唯一可行方案是**在 DSLX 层面手动构建状态机**，将乘法操作封装在 proc 中，通过显式状态推进实现多周期计算。代码清单 17 展示了这一模式：

```
pub proc multicycle_multiplier {
    a_r: chan<s32> in;
    b_r: chan<s32> in;
    result_s: chan<s64> out;

    config(a_r: chan<s32> in, b_r: chan<s32> in,
           result_s: chan<s64> out) {
        (a_r, b_r, result_s)
    }

    init { (s32:0, s32:0, u2:0) }  // (a, b, state)

    next(state: (s32, s32, u2)) {
        let (a_reg, b_reg, phase) = state;
        let tok = join();

        match phase {
            u2:0 => {  // IDLE: 接收输入
                let (t, a) = recv(tok, a_r);
```

```
                let (t, b) = recv(t, b_r);
                (a, b, u2:1)
            },
            u2:1 => {  // COMPUTE: 启动乘法
                let _ = a_reg * b_reg;
                (a_reg, b_reg, u2:2)
            },
            u2:2 => {  // OUTPUT: 输出结果
                let result = (a_reg as s64) * (b_reg as s64);
                let t = send(tok, result_s, result);
                (s32:0, s32:0, u2:0)
            },
            _ => (s32:0, s32:0, u2:0)
        }
    }
}
```

Listing 17: 手动实现的多周期乘法状态机

这一方案的本质是**将调度器的职责部分上移至程序员**：程序员通过显式状态编码控制”何时启动乘法”和”何时采集结果”。该方案虽然可行，但存在明显缺陷：代码侵入性高、资源效率低、可移植性差（延迟假设硬编码在状态机中）。

5.6.4 黑箱模型的根本局限

XLS 无法自动拆分多周期操作的深层原因，在于其**黑箱延迟模型**的设计哲学。XLS 将操作视为具有延迟属性的抽象原语，但**完全不暴露其内部结构**。乘法器被建模为”输入 a, b → 延迟 D → 输出 result”的黑箱，而非”部分积生成 → 华莱士树压缩 → 最终加法”的结构化描述。

这种抽象的优势在于工艺无关性和实现灵活性——Yosys 可以根据工艺库选择 Booth 编码、阵列乘法或 DSP 宏单元。但其代价是**调度器丧失了操作内并行性**：

- 无法将部分积生成与压缩树分配到不同 stage；
- 无法自动插入 pipeline 寄存器到乘法器内部；
- 无法基于结构信息优化跨操作边界的数据流。

5.7 有状态黑盒与 SRAM 封装问题

比多周期组合逻辑更深层的问题，是**有状态黑盒**（Stateful Black-Box）的不可封装性。以 SRAM 为例：虽然其接口看起来可以用“地址 → 固定延迟 → 数据”的黑箱轮廓描述，但 SRAM 内部存储阵列的状态对 XLS 完全不可见。这意味着：

- 编译器无法进行地址别名分析，也就不能安全地调度独立的读写操作；
- Z3 SMT 等价性检查器和 IR Fuzzer 无法验证涉及该黑盒的优化是否正确；

- ‘init‘与状态初始化语义无法映射到 SRAM 的 X 态或外部加载行为。

因此，`extern_verilog` 和 `dummy_block` 目前**不适合**用来封装 SRAM。GitHub 上的增强型 issue #861 “RAM Support” 已经明确了官方路线：不通过黑盒导入 SRAM，而是构建 **native RAM model → lowering pass → dedicated codegen** 的完整链路。截至目前，1RW/1R1W RAM 的 model、lowering 和 codegen 已经完成，但 2RW 等复杂变体仍在开发中。工程上更务实的做法是将 SRAM 放在 XLS 模块外部，通过 `chan<T>`的请求/响应接口与之通信。与此同时，社区也在探索更高层的张量抽象：开放 issue #3983 提议在 XLS IR 中引入 `MatMul` 一级操作，并配套组合逻辑全展开、流水线 MAC 树、甚至 Systolic Array mapping 等 lowering 策略，显示出向 ML 加速器领域扩张的野心。

5.8 局限与未来方向

除了上述具体技术边界外，XLS 的黑箱方法论还存在以下待改进空间：

- **迭代精化**：通过实际综合反馈调整延迟估计，形成闭环优化；
- **技术映射**：在 XLS 内部引入标准单元映射层，实现更精细的面积控制；
- **动态调度**：当前仅支持静态调度，未来可能探索基于运行时条件的流水线调整；
- **操作展开**：将乘法等复杂操作在 IR 层面展开为显式部分积网络，允许调度器跨 stage 分配；
- **延迟标注**：允许用户为操作标注 $\delta > 1$ 的延迟，由调度器自动插入 stage 间寄存器；
- **外部 IP 集成**：通过延迟合约机制接入外部设计的流水线化乘法器黑箱原语。

尽管如此，XLS 的黑箱哲学代表了 HLS 领域的重要演进方向。

6 XLS 与 Chisel/Chipyard：设计范式的对比

在当代开源硬件设计工具生态中，UC Berkeley 开发的 Chisel 语言和 Chipyard 框架是与 XLS 最具可比性的项目。两者都致力于提高硬件设计效率，但采用了根本不同的方法论路径。深入理解这种差异，有助于在实践中做出恰当的技术选型。

6.1 设计哲学：对象构造 vs 数据流编译

Chisel（Constructing Hardware In a Scala Embedded Language）将硬件设计视为**面向对象的软件构造问题**——开发者使用 Scala 编写程序，该程序在执行时” 构造” 出硬件电路描述。这种方法继承了软件工程的成熟范式：参数化、继承、多态、元编程。

XLS 则采用**数据流编译**的路径——开发者描述计算的数据流图，编译器通过调度算法自动将其映射到时序硬件。这种方法建立在 Kahn 进程网络的形式化语义之上，强调确定性、静态分析和自动化优化。

表 5 从多个维度对比了两种设计范式：

表 5: Chisel/Chipyard 与 XLS 的设计范式对比

维度	Chisel/Chipyard	XLS
计算模型	事件驱动/时钟精确	Kahn 进程网络/数据流
时间抽象	显式时钟周期	隐式同步（Token 驱动）
设计层次	RTL 级（寄存器、线网）	算法级（函数、proc）
优化策略	结构变换（FIRRTL）	调度+代数优化
生成方式	程序构造电路	编译器综合电路
编程语言	Scala（宿主语言）	DSLX（专用数据流语言）

6.2 语言抽象层次对比

6.2.1 类型系统

Chisel 的类型系统基于 Scala，支持完整的泛型编程：

```
// Chisel: 参数化位宽和类型
def createAdder[T <: Data](gen: T): Module = {
  new Module {
    val io = IO(new Bundle {
      val a = Input(gen.cloneType)
      val b = Input(gen.cloneType)
      val sum = Output(gen.cloneType)
    })
    io.sum := io.a + io.b
  }
}
```

XLS 的 DSLX 类型系统专注于硬件友好的位宽处理：

```
// DSLX: 任意位宽类型与算术推断
fn add<N: u32>(a: uN[N], b: uN[N]) -> uN[N+1] {
  a + b  // 结果位宽自动推断为N+1
}
```

Chisel 的泛型系统更强大，支持复杂的参数化设计空间探索（这是 Chipyard 的核心能力）；XLS 的位宽处理更灵活，支持任意精度算术。

6.2.2 时序建模

Chisel 采用显式时序建模：

```
// Chisel: 显式寄存器声明和时序逻辑
class Counter extends Module {
  val reg = RegInit(0.U(8.W))  // 显式创建寄存器
  when(io.en) {
    reg := reg + 1.U           // 显式描述下一状态
```

```
  }
}
```

XLS 采用隐式时序抽象：

```
// DSLX: 状态抽象为next函数
proc counter {
    init { u8:0 }
    next(state: u8) {
        let (tok, en) = recv(join(), en_r);
        if en { state + u8:1 } else { state }
    }
}
```

显式时序（Chisel）给予开发者完全的控制权，适合精细优化；隐式时序（XLS）释放开发者从手动流水线设计的负担，编译器自动处理时序约束。

6.2.3 并发与通信

Chisel/Chipyard 采用**共享内存+缓存一致性**模型，通过 TileLink 总线协议实现模块间通信：

```
// Chisel: TileLink总线接口
val tlBus = TileLinkBundle(params)
// 处理器核心通过缓存一致性协议访问共享内存
```

XLS 采用**消息传递+数据流**模型，通过 channel 实现确定性通信：

```
// XLS: channel通信
proc producer {
    output_s: chan<u32> out;
    next(state: u32) {
        let tok = send(join(), output_s, state);
        state + u32:1
    }
}
```

共享内存模型适合复杂 SoC 设计，支持动态数据访问模式；消息传递模型适合加速器内核设计，保证确定性行为并简化形式化验证。

6.3 编译流程与优化策略

Chisel 的编译流程体现了”软件构造”的哲学：

```
Chisel Scala --> FIRRTL --> 变换Passes --> Verilog --> 综合
      |            |             |                       |
      v            v             v                       v
Scala执行     宽度推断     常量传播/模块内联       工艺映射
 构造电路    连接解析      寄存器优化              布局布线
```

FIRRTL（Flexible Intermediate Representation for RTL）是 Chisel 的核心创新，它保留了模块层次结构，支持模块内联、宽度推断、常量传播等变换。然而，时序优化（尤其是流水线化）通常需要手动设计或使用黑盒 IP。

XLS 的编译流程体现了"数据流综合"的哲学：

```
DSLX --> XLS IR --> 优化Passes --> 调度 --> Verilog --> 综合
   |        |          |            |            |
   v        v          v            v            v
类型检查 Sea-of-Nodes 位窄化/    SDC/Min-Cut   工艺映射
位宽推断 单一IR       强度削弱/   自动流水线化
                      选择优化
```

XLS 的核心创新在于**调度器**——它将数据流图视为带延迟属性的节点集合，使用 SDC（System of Difference Constraints）或 Min-Cut 算法自动确定流水线切分点。这与 Chisel 形成鲜明对比：Chisel 要求开发者显式描述流水线寄存器，而 XLS 通过延迟模型自动决定操作分组。

考虑三连加运算 `result = a + b + c + d` 的优化：

Chisel 实现：

```scala
val sum1 = io.a + io.b  // 第一级加法
val sum2 = sum1 + io.c  // 第二级加法（依赖sum1）
val result = sum2 + io.d // 关键路径：3个加法器延迟

// 优化需要手动重结合：
val sum_left = io.a + io.b
val sum_right = io.c + io.d
val result = sum_left + sum_right  // 关键路径：2个加法器延迟
```

XLS 实现：

```
let sum1 = a + b;
let sum2 = sum1 + c;
let result = sum2 + d;
// XLS的arith_simplification_pass自动将线性链转换为平衡树
```

6.4 适用场景与应用生态

Chisel/Chipyard 的优势场景：

- **通用处理器设计**：Rocket Chip（RISC-V 处理器）和 BOOM（乱序超标量核心）证明了 Chisel 在复杂 CPU 设计中的能力；
- **SoC 集成**：Chipyard 提供完整的 SoC 生成框架，支持处理器、加速器、内存系统的参数化集成；
- **设计空间探索**：通过 Scala 配置对象，可以快速生成数百种 SoC 变体进行评估；

- **敏捷验证**：FireSim 提供 FPGA 加速的周期精确仿真，支持大规模软硬件协同验证。

XLS 的优势场景：

- **数据流加速器**：AI 推理（GEMM、卷积）、信号处理（FFT、FIR）、加密/哈希等计算密集型内核；
- **自动流水线设计**：编译器自动处理时序约束和流水线寄存器插入，减少手动优化负担；
- **确定性计算**：KPN 语义保证软硬件行为等价，简化形式化验证；
- **位级优化**：位窄化、强度削弱等优化 pass 针对硬件特性深度优化。

6.5 互补性分析与未来融合

Chisel/Chipyard 和 XLS 并非竞争关系，而是**面向不同抽象层次的互补工具**。理想的协同工作流程是：

1. 使用 XLS 设计数据流加速器内核（如矩阵乘法、卷积引擎）；
2. 使用 Chisel/Chipyard 设计处理器核心和 SoC 基础设施；
3. 通过标准接口（TileLink 或 AXI）将 XLS 生成的加速器集成到 Chisel SoC 中；
4. 利用 FireSim 仿真完整的异构系统。

这种分工充分发挥了两种工具的优势：XLS 负责算法级数据流优化和自动流水线生成，Chisel/Chipyard 负责系统级集成和复杂控制逻辑。随着开源硬件生态的成熟，两种工具链的互操作将成为敏捷芯片开发的重要支撑。

7 XLS 应用生态与社区研究

本节对公开已知使用或集成 Google XLS（及其前端语言 DSLX）的项目、组织与学术工作进行系统性梳理，补充本文在“生态系统”维度的分析。

7.1 CHIPS Alliance / silicon-notebooks：教育流片一体化

CHIPS Alliance（Linux Foundation 下的开源硬件联盟）维护的 `silicon-notebooks` 仓库（174 stars）通过 Jupyter / Google Colab 笔记本降低开源硅片设计门槛，提供从代码到 GDS 的完整交互式教程。

silicon-notebooks 明确包含两份 XLS 相关 notebook：

- `xls-adder-openlane.ipynb`：使用 XLS HLS 设计简单的 1 位加法器，再通过 **OpenLane** RTL→GDS 流程 targeting SKY130 PDK。该 notebook 被 Google for Developers Silicon 官方页面直接引用。

- `xls-workshop-openlane.ipynb`：口号为 *From code to silicon using XLS and OpenLane*，展示了如何借助 **Google OpenMPW shuttle** 将设计实际制造为芯片。

其价值在于：零安装门槛（Colab 云端运行）、端到端可视（DSLX → IR → Verilog → OpenLane → GDS），以及结合 SKY130 / GF180MCU 两种免费工艺节点的实际流片体验。CHIPS Alliance 的 silicon-notebooks 是目前最系统、最面向初学者的 XLS 教育资料。

7.2 Antmicro：XLS 与压缩算法的工业级验证

Antmicro 是一家专注于开源 EDA、嵌入式系统及 RISC-V 生态的工程咨询公司，与 Google 在 XLS 领域有深度合作。

2023 年，Antmicro 在博客中展示了如何用 XLS / DSLX 实现 **Run-Length Encoding (RLE)** 和 **Dictionary Based Encoding (DBE)** 两种压缩算法硬件加速器，证明 XLS 能够处理具有一定控制逻辑的流媒体数据路径。

2025 年，Antmicro 发布了更完整的 **Zstandard (ZSTD)** 解码器硬件模块，这是目前公开报道中**最复杂的 XLS 应用案例之一**。其关键特点包括：

- 利用 XLS 的 "软件风格" 高层次描述能力，将算法架构与底层 RTL 细节解耦；
- 结合 XLS 的形式化验证和 Fuzzing 工具链，对复杂状态机进行高覆盖率验证；
- 标志着 XLS 从概念验证向实用压缩/存储加速器迈进。

Antmicro 还开源了 `antmicro/xls-cosimulation-riscv-firmware` 和 `antmicro/xls-cosimulation-demonstrator` 仓库，构建包含 **VexRiscV CPU** 与 **XLS RLE encoder** 的协同仿真系统，利用 Renode 仿真平台运行 RISC-V 固件并与 XLS 生成的硬件外设联合验证。这些工作代表了 XLS 在工业算法加速方向的最前沿探索。

7.3 Cornell-zhang / Allo：学术研究中的 XLS ASIC Backend

Allo 是 Cornell 提出的 Python 嵌入式加速器设计语言（ADL），基于 MLIR，强调模块化与可组合性。Allo 官方文档中设有专门的 "Google XLS (ASIC)" 页面，详细介绍了如何将 Allo 描述的高层算子编译到 XLS 工具链：

- **硬件生成**：`s.build(target="xlscc", use_memory=True)` 生成 XLS 的 C++ 前端代码，可直接进入 XLS 综合流程；
- **软件仿真**：`mode="sw_emu"` 允许先在 CPU 上进行 C 级仿真验证，再切换硬件生成；
- **内存接口映射**：通过 `Memory` 类注解自动生成 `rewrites.textproto`，将数组映射到 XLS 支持的 RAM_1RW 和 RAM_1R1W 等存储类型。

Cornell CS 6120（Advanced Compilers）课程的学生项目 "From Allo to XLS" 实现了三条编译通路：`mlir_to_dslx_fn.py`（MLIR → DSLX `fn`）、`mlir_to_ir_fn.py`（MLIR → XLS IR）以及 `mlir_to_verilog_proc.py`（MLIR → DSLX `proc` → Verilog）。其核心技术挑战包

括命令式到函数式的翻译（每次 `store` 转换为 DSLX 的 `update` 表达式返回新数组）、多维数组线性化，以及从展开后的 PE 函数中重建 systolic array 空间网格结构。

表 6 展示了该项目基于 SKY130 / OpenROAD 的 PPA 实验结果（2×2 place-and-route）：

表 6: Allo-XLS Backend PPA 实验结果（SKY130）

设计	类型	级数	面积 (μm^2)	周期	频率	功耗	延迟
Combinational GEMM	uint32	5	36,480	4.06 ns	∼246 MHz	17.38 mW	5 cycles
Systolic Array	uint32	2	51,076	4.10 ns	∼244 MHz	16.74 mW	18 cycles

Systolic Array 的扩展实验进一步表明：XLS 的 `proc` 网络需要 **≥2 级流水线** 才能成功生成 Verilog，单级设计会失败；随着阵列扩大（4×4），systolic array 的周期数增长可控，但 **深度 ≥3 时 XLS 调度器会超时**（1480-cycle timeout），说明当前工具链在大型 `proc` 网络上仍有鲁棒性瓶颈。此外，XLS backend 目前 **仅支持整数与定点类型**，浮点类型尚未打通。

7.4 TinyTapeout 社区：XLS 的实际流片验证

TinyTapeout 是一个多项目晶圆（MPW）众包平台，极大降低了个人和小团队的定制芯片门槛。Google 员工 Johan Euphrosine（GitHub: proppy）是其中最活跃的 XLS 早期实践者之一。根据 TinyTapeout 官方批次统计以及对各仓库 `info.yaml` 的溯源，**真正被提交并实际流片的 XLS 设计仅有以下 2 个 unique 设计**（见表 7）：

表 7: TinyTapeout 已确认流片的 XLS 设计

项目	功能	流片批次	技术特点
`tt02-xls-counter`	8 位计数器	TT02 #051, TT03 #141	`proc` 状态机，含 `send` 通道
`tt02-xls-popcount`	位 popcount	TT02 #042, TT03 #132	纯组合逻辑，bit-parallel 算法

这两个设计是目前**唯一公开可验证的 XLS 实际流片实例**，分别覆盖了 DSLX 的带状态 `proc` 抽象和纯组合函数抽象。它们的 official chip 页面均提供了 GDS 渲染图和 OASIS 3D viewer 链接，是普通开发者体验 XLS 在真实 silicon 上成功的最低成本证明。

7.5 其他社区生态：xlsynth、dslx-llm 与 FLUENT

xlsynth 生态系统 是目前最活跃的 XLS 社区组织之一，围绕 Google XLS 构建了多语言绑定、Bazel 构建规则、可复用 IP 库和 Verilog 拼接工具等完整工具链（见表 8）。

`xlsynth/dslx-llm` 则专门为大语言模型提供 DSLX 训练数据、提示词模板和自动评测工具。该项目明确主张 DSLX 比 Verilog 更适合 LLM 生成——XLS IR 语义完全确定，避免了传统 RTL 工具链中大量实现定义/未定义行为；同时 DSLX 语法接近 Rust，便于 LLM 迁移软件领域的先验知识。

FLUENT（`encryptogroup/FLUENT`）是一个用于半私有函数求值（SPFE）的研究框架，其代码库中嵌入了扩展版 XLS，并直接调用 `ir_converter_main`、`opt_main`、`codegen_main` 等 XLS 二进制工具完成从高层描述到硬件的转换。这是公开仓库中少数将 XLS 作为**内部编译基础设施**而非单纯示例引用的学术研究项目。

表 8: xlsynth 生态核心项目

项目	定位	链接
`xlsynth/xlsynth`	社区 release fork，提供多平台预编译共享库	`github.com/xlsynth/xlsynth`
`xlsynth/xlsynth-crate`	Rust 工作空间（FFI、API、CLI、门数估计）	`github.com/xlsynth/xlsynth-crate`
`xlsynth/rules_xlsynth`	下一代 Bazel 构建规则	`github.com/xlsynth/rules_xlsynth`
`xlsynth/bedrock-rtl`	高质量 SystemVerilog RTL 库	`github.com/xlsynth/bedrock-rtl`
`xlsynth/bedrock-dslx`	可复用 DSLX 库（早期阶段）	`github.com/xlsynth/bedrock-dslx`
`xlsynth/topstitch`	基于 Rust 的 Verilog 模块层级拼接工具	`github.com/xlsynth/topstitch`

WOSET 2021 论文 *Porting software to hardware using XLS/DSLX*（作者 Johan Euphrosine & Rob Springer）系统阐述了如何将 C 语言例程重写为 DSLX，并展示 XLS 在 FPGA 和 ASIC 流程中的集成方式，包括在 **Lattice ICE40UP5K** 开发板上的端到端开源 FPGA 综合实例。

7.6 Google 内部使用与 Silicon 官方生态

XLS README 明确指出：*XLS enables the rapid development of hardware IP that also runs as efficient host software*，这是 Google 为解决软硬件协同而开发的内部工具链。但官方也标注其为 experimental 状态，现阶段 DSLX 改进可能不考虑向后兼容。

Google for Developers Silicon 页面（`developers.google.com/silicon`）设有 Notebooks 专区，直接链接到 CHIPS Alliance 的 `xls-adder-openlane.ipynb`，说明 Google 将 XLS 视为其 OpenMPW shuttle 教育体系中的核心 HLS 工具之一。

Chris Leary 在 UCSC 2024 客座演讲及博客中明确指出，XLS 已经在 Google 的 **实际产品** 中使用，且内部采用团队范围很广（“run the gamut inside the company”），涵盖了从算法工程师到硬件工程师的跨域协作。

与 OpenTitan 的关系：OpenTitan（Google 主导的开源硅片 Root of Trust 项目）目前无公开证据表明其核心 RTL 由 XLS/DSLX 生成；它主要使用 SystemVerilog。两者同属 Google 的开源硬件战略，但在技术栈上保持独立。OpenTitan 走的是经过形式化验证的“传统 RTL”路线，而 XLS 走的是“高层次综合 + 数据流模型”的新路线。

7.7 生态成熟度评估

表 9 对 XLS 各主要采用者进行了成熟度与代表性总结：

Google XLS 的公开采用者尚不如 Chisel/Rocket Chip 生态庞大，但已经形成了 **“教育 → 学术研究 → 社区流片 → 工业算法加速”** 的完整梯度。对于考虑采用 XLS 的团队，建议的参考路径是：先用 **silicon-notebooks + TinyTapeout** 做小模块流片验证，再逐步扩展到算法加速器或像 Allo 那样作为学术后端进行探索。

表 9: XLS 生态成熟度评估

采用者/领域	成熟度	关键产出	代表性链接
CHIPS Alliance	3/5	教育 notebook、加法器教程	`github.com/chipsalliance/silicon-notebooks`
Antmicro	4/5	RLE/DBE/ZSTD 加速器、协同仿真	`antmicro.com/blog/2025/04/building-a-zstd-decoder-in-xls`
Cornell Allo	3/5	GEMM→XLS backend、学术研究	`cornell-zhang.github.io/allo/backends/xls.html`
TinyTapeout	4/5	实际流片芯片（counter、popcount）	`github.com/proppy/tt02-xls-counter`
xlsynth	2/5	Rust 绑定、LLM 训练素材	`github.com/xlsynth/xlsynth`
Google 内部	4/5（推断）	内部加速器 IP、OpenMPW 支持	`github.com/google/xls`

7.8 GitHub Discussions 社区声音分析

除上述项目和组织外，XLS 官方仓库的 GitHub Discussions 是观察普通开发者使用痛点与技术关注点的直接窗口。截至 2026 年 4 月，该仓库共有 **26 个**公开 Discussion。本节基于对全部讨论的系统爬取与分类，梳理社区关注的核心主题。

表 10 按主题领域对这 26 个讨论进行了归纳。需要指出的是，XLS 社区的 Discussion 活跃度远低于其 Issue 区，且多数帖子为技术性 Q&A，回复量普遍较小；但高回复帖子（如 systolic array 实现、one-hot-select 语法糖）往往精准反映了工程落地的关键障碍。

7.8.1 构建与入门障碍

社区中约有 11

7.8.2 DSLX 语法与语义摩擦

DSLX 作为一门新生的硬件设计语言，其语法细节和边缘语义引发了较多困惑。例如，#1494 对“越界数组切片会 clamp 到最后一个元素”的语义表示惊讶——这与 Python 抛出异常或 C 产生未定义行为均不同，是 XLS 为了保证硬件确定性而特意设计的“安全饱和”语义，但对软件背景用户并不直观。

#2002 则聚焦于 one-hot-select 的语法糖需求：开发者希望用布尔数组便捷地从复杂类型的数组中做选择，而不必手动展开成大量 ‘if-else‘。该讨论获得了 9 条回复，反映出 DSLX 在高层次抽象表达上仍有提升空间。类似地，#2968 询问是否存在类似 ‘flip()‘ 的语法来自动反转 channel struct 的方向，亦暴露出当前 DSLX 在接口可组合性上的工具缺口。

表 10: XLS GitHub Discussions 主题分类汇总（截至 2026-04）

主题领域	代表性讨论标题	数量	最高回复
构建与环境配置	Ubuntu 18.04 / macOS 构建报错、Chatroom 询问	3	4
DSLX 语言语义	‘slice‘ 用法、越界 clamp 语义、one-hot-select 语法糖	5	9
Channel / Proc 通信模型	peek channel、多 IO 操作、互斥分析粒度、FIFO 深度	6	21
代码生成与 RTL 质量	Verilog 子模块实例化、整数乘法流水线化、优化后 proc 为空	4	5
形式化验证与 Z3	IR 增加 FV、mod 不支持、Z3 重复变量声明	3	2
硬件设计实践	Systolic Array 实现、NN feedforward 吞吐优化	2	21
工具链基础设施	新 delay model、xlsynth-g8r QoR 分析提案	2	5

7.8.3 Channel / Proc 通信模型：最大的痛点域

涉及 channel 与 proc 的讨论数量最多（6/26，23

- **“Peek but not consume”**（#3478）：开发者希望在 RTL 层面只读 ‘valid‘ 而不置 ‘ready‘，对应非破坏性查看 FIFO 队首。但 XLS 当前 proc 语义中不存在 ‘peek‘ 原语，因为这会破坏 KPN 的 “无界 FIFO + 阻塞 recv” 抽象一致性。
- **同一 channel 的多重 IO 操作**（#3736，#3852）：这两个讨论分别获得了 6 和 2 条回复，探讨如何在同一个 proc 的 ‘next‘ 函数中对同一个 channel 进行多次 ‘recv‘ 或 ‘send‘。当前编译器对此有严格的合法性检查，某些写法会导致调度失败或必须手动拆分到不同状态。
- **互斥性分析粒度**（#1996）：一位社区成员深入阅读源码后提问——为什么编译器在判断 channel 操作是否互斥时，是按整个 proc 分析，而不是按每个 channel 在每个 proc 中分别分析。这触及了 XLS 调度器内部实现的工程折中，回复的结论是：当前分析算法为了保守正确性而采取的粗粒度策略。
- **不同 proc 间共享硬件资源**（#1906）：在 4×4 PE 阵列设计中，开发者询问多个 proc 实例是否允许共享同一个乘法器等硬件单元。这本质上是在 XLS “静态实例化” 语义下讨论面积优化的可能性，回复的结论是：XLS 目前不会在 IR 层面自动跨 proc 复用硬件资源，面积冗余需由开发者通过参数化设计或外部脚本控制生成实例数来规避。

7.8.4 代码生成质量与流水线 Limitation

#434 和 #646 分别从相反方向质疑了 RTL 生成质量：前者抱怨 ‘simple_add‘ 示例生成的 Verilog“看起来未优化”（含有大量冗余线网和内联逻辑），后者则提议将 DSLX 的 ‘fn‘ 调用

lower 为独立的 Verilog module 实例化，以便提高代码可读性和支持层次化综合。这两个帖子遥相呼应，说明当前 XLS 的 codegen 在 “优化至上” 与 “生成可维护 RTL” 之间存在张力。

#3065 是一个极具代表性的讨论：开发者明确询问 “Google XLS 能否将整数乘法流水线化”，这与本文第 4.3 节及第 5.6 节的分析完全一致。讨论中的官方回复确认了 XLS 当前无法自动将单一乘法操作拆分到多个 pipeline stage，因为其黑箱延迟模型假设每个 IR 节点是原子性的。普通用户在实际开发中确实会立即撞上这一方法论边界。

#2500 则展示了优化器的另一侧问题：运行 ‘matmul_4x4‘ 示例后，优化后的顶层 proc 意外变成了空过程。该问题反映了死代码消除（DCE）和 proc-level 优化在某些边界条件下的激进行为，对初学者而言极具迷惑性。

7.8.5 形式化验证链路的盲区

尽管 XLS 将形式化验证和 Fuzzing 作为核心卖点加以宣传，但社区讨论暴露出其 FV 基础设施仍存在明显缺口：

- **Z3 不支持 mod 操作**（#1670）：‘UMod‘ / ‘SMod‘ 在 IR 到 Z3 SMT 的翻译中尚未实现，导致任何包含取模运算的设计都无法走通形式化验证流程。
- **IR→Z3 的重复变量声明**（#2293）：‘assert¡ 宏的 lowering 会在 IR 中引入特定节点模式，而 Z3 转换器对此处理不当，生成重复绑定的 SMT 变量，导致验证失败。
- **IR 级额外 FV 的需求**（#1537）：社区成员提议在 DSLX→IR→ 优化 IR→Verilog 的流程中增设额外的形式化验证检查点，说明当前端到端验证仍不能让所有用户满意。

这些问题并不影响 XLS 解释器/Fuzzer 对常规功能的覆盖，但对于需要高保证安全性的设计而言，它们指出了 FV 工具链仍需弥合的覆盖率差距。

7.8.6 加速器设计实践：Systolic Array 与神经网络

在所有讨论中，热度最高的当属 #1663 *How to implement Systolic Array in DSLX*（21 条回复）。该帖涉及向量类型在 xlscc 中的支持、channel 数组的参数化、数据注入与脉动时序的对齐等系统工程问题。回复中既有官方维护者的技术澄清，也有社区成员分享的实践代码，是目前公开网络上关于 XLS 脉动阵列实现最密集的对话集合。其高热度本身就说明了：Systolic Array / GEMM 加速器是吸引开发者接触 XLS 的核心动力，也是最能体现 DSLX 表达能力上限的试金石。

另一个 “Show and tell” 类型帖子 #2919 分享了用 DSLX 实现神经网络 jet tagging 分类器的前馈 proc 网络，并测量了其延迟与吞吐。这代表了 XLS 在 AI 加速器领域的实际应用场景，虽然回复仅 1 条，但其分享的脚本是少有的完整工程案例。

7.8.7 关键洞察

综合对全部 26 个 GitHub Discussions 的文本分析，可以提炼出以下五点洞察：

1. **KPN / Channel 语义是工程落地的最大瓶颈**。peek、多 IO 操作、互斥分析粒度、FIFO 深度配置等问题反复出现，说明 Kahn 进程网络的“无界 FIFO”抽象与物理硬件的有限缓冲、可观测信号（valid/ready）之间存在尚未完全弥合的语义鸿沟。

2. **Systolic Array / MAC 是社区最关心的设计目标**。#1663 以 21 条回复遥遥领先，GEMM、神经网络加速器的讨论贯穿多个帖子，显示 XLS 在 AI 加速器领域具有强烈的吸引力，同时也面临可扩展性和调度器鲁棒性的实际挑战。

3. **黑箱模型的原子性限制已被用户实际感知**。#3065 直接证实了开发者会在乘法流水线化场景中撞到黑箱延迟模型的边界，与我们前文对多周期操作限制的分析互为佐证。

4. **R_codegen 的可控性与可维护性受质疑**。从子模块实例化到端口命名再到 trivial 设计的冗余逻辑，社区对“生成什么样的 RTL”抱有与“生成功能正确 RTL”同等重要的期待，这或许暗示了 XLS 在未来需要更多 codegen-quality 相关的用户侧配置选项。

5. **形式化验证链路存在明显盲区**。Z3 转换器对 ‘mod‘ 和重复声明的支持缺失，以及 IR 级额外 FV 的需求，说明当前 FV 基础设施尚未达到“默认启用”的成熟度。

8 总结与展望

Google XLS 代表了一种从语言到编译器、从 IR 到 RTL 的系统性 HLS 方法论革新。它通过引入 DSLX 数据流语言，将冯 · 诺依曼计算模型的包袱抛诸脑后，以 Kahn 进程网络为语义基础，实现了硬件描述与软件执行在共享源代码上的统一。其单一 Sea-of-Nodes IR、Token 驱动的隐式同步、以及深入的位级优化和数学化调度算法，构成了一个功能完备且已实际应用于 Google 内部项目和开源社区（如 CERN 的 hls4ml 集成计划 [7]）的工具链。

然而，XLS 目前仍处于实验快速发展阶段。其局限包括：DSLX 作为一门新语言，学习与生态迁移成本较高；xlscc（C++前端）目前仅由贡献者 best-effort 维护，功能与稳定性尚不及 DSLX；对于复杂控制流（如深层嵌套循环与不规则内存访问模式）的表达与优化，仍有较大提升空间。未来，随着社区在 hls4ml、Allo 等框架中的集成探索不断深入，XLS 有望成为后摩尔时代专用加速器快速原型设计的重要基础设施。

致谢

本文写作过程中使用了 Claude AI 助手进行资料整理与分析。

参考文献

[1] Xilinx. *Vivado Design Suite User Guide: High-Level Synthesis*. UG902, 2023.

[2] G. Kahn. The Semantics of a Simple Language for Parallel Programming. In *Proc. IFIP Congress*, 1974: 471–475.

[3] 体系结构领域相关综述. 数据流架构与粗粒度可重构架构的发展历程. 知乎专栏, 2021.

[4] Google. XLS: Accelerated HW Synthesis. GitHub Repository, `https://github.com/google/xls`, 2020.

[5] C. Leary. XLS (Accelerated HW Synthesis) Lecture @ UCSC CSE228A — Winter 2024. `https://blog.cdleary.com/posts/2024/04/08/xls-accelerated-hw-synthesis-at-ucsc-2024/`, 2024.

[6] Google. XLS Documentation: DSLX Reference, IR Overview, and Scheduling. `https://google.github.io/xls/`, 2020–2025.

[7] V. Loncar and D. Danopoulos. Integrating Support for Google XLS in HLS4ML. CERN Google Summer of Code Proposal, 2025.

[8] A. Tretter et al. Executing Dataflow Actors as Kahn Processes. In *Proc. ESWEEK*, 2015, pp. 1–10.

[9] J. R. Gurd, C. C. Kirkham, and I. Watson. The Manchester Prototype Dataflow Computer. *Communications of the ACM*, vol. 28, no. 1, pp. 34–52, 1985.

[10] E. A. Lee and T. M. Parks. Dataflow Process Networks. *Proceedings of the IEEE*, vol. 83, no. 5, pp. 773–801, 1995.

[11] G. Kahn and D. B. MacQueen. Coroutines and Networks of Parallel Processes. In *Information Processing*, 1977, pp. 993–998.

[12] C. Hewitt, P. Bishop, and R. Steiger. A Universal Modular ACTOR Formalism for Artificial Intelligence. In *Proc. IJCAI*, 1973, pp. 235–245.

[13] J. Bachrach et al. Chisel: Constructing Hardware in a Scala Embedded Language. In *Proc. DAC*, 2012, pp. 1216–1225.

[14] UC Berkeley. Chipyard: Agile RISC-V SoC Design Framework. GitHub Repository, `https://github.com/ucb-bar/chipyard`, 2024.

[15] A. Izraelevitz et al. Reusability is FIRRTL Ground: Hardware Construction Languages, Compiler Frameworks, and Transformations. In *Proc. ICCAD*, 2017, pp. 209–216.

[16] S. Karandikar et al. FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud. In *Proc. ISCA*, 2018, pp. 29–42.